ACTIVIDAD

Dosier de Actividades del Tema: *Optimizar y documentar el código*

DAM / DAW

Módulo 5 Entornos de desarrollo

Linkia FP

EJERCICIO 1

En esta actividad aplicaremos algunos de los **patrones de refactorización** vistos en clase para optimizar el código.

Para cada bloque de código adjunto abajo:

1. **Crea una nueva clase** Java con el código (adjunta la evidencia)
2. Refactoriza el código
3. Adjunta el código una vez refactorizado (adjuntar evidencia)
4. **Explica** a continuación del código refactorizado **qué patrón has usado** y **comenta los cambios** que se han hecho**.**

Bloques de código a utilizar en la actividad:

Bloque 1: Tenemos una clase que tiene un atributo que necesita información y funciones propias. Qué patrón de refactorización tenemos que usar para añadir otra información del cliente como edad, dni, etc.

```
public class Pedido {
   private int id;
   private String cliente;

   public Pedido(int id,String cliente) {
      this.id = id;
      this.cliente = cliente;
   }
}
```

Al observar el código original, vemos que la clase Pedido contiene un atributo String cliente, lo cual a simple vista puede parecer suficiente si solo queremos guardar un nombre. Pero en cuanto se nos pide añadir más información del cliente (como la edad, el DNI, etc.), este diseño se queda corto. Usar un String no nos permite crecer ni estructurar la información correctamente.

Para solucionar este problema y mejorar el diseño, decidí aplicar el patrón de refactorización llamado "***Reemplazar Datos y Valores por Objetos***". Este patrón se usa cuando un dato simple empieza a necesitar más información o comportamiento propio. En este caso, tiene mucho más sentido que cliente sea un objeto, no un String.

Gracias a esta refactorización, ahora Pedido puede trabajar con una clase Cliente que encapsula todos los datos necesarios. Esto hace que el código sea más limpio, modular y fácil de mantener o ampliar en el futuro.

Adjunto código:

```java
*Cliente.java   *Pedido.java   *Main.java
package entornos.ejercicio1;

/**
 * Clase que representa un Pedido, que contiene un ID y un Cliente asociado.
 */
public class Pedido {
    private int id;
    private Cliente cliente;

    /**
     * Constructor de la clase Pedido.
     * @param id Identificador del pedido.
     * @param cliente Cliente que realiza el pedido.
     */
    public Pedido(int id, Cliente cliente) {
        this.id = id;
        this.cliente = cliente;
    }

    public int getId() {
        return id;
    }

    public Cliente getCliente() {
        return cliente;
    }
}
```

Bloque 2: Qué patrón tenemos que usar para refactorizar una clase que hace el trabajo que debería ser hecho por dos clases.

```java
public class Cliente {

   private String nombre;
   private String telefonoTrabajo;
   private String prefijoTelefonoTrabajo;
   private String telefonoCasa;
   private String prefijoTelefonoCasa;

   public String getTelefonoTrabajo() {
      return telefonoTrabajo;
   }
   public String getTelefonoCasa() {
      return telefonoCasa;
   }
}
```

Una vez que decidimos que cliente debía ser algo más que un simple String, la solución lógica fue crear una nueva clase que representara de forma adecuada a una persona. Es decir, si un cliente va a tener nombre, edad, DNI (y posiblemente más

información en el futuro), lo más natural es encapsular todo eso dentro de una clase propia.

Aquí es donde entra la clase Cliente, que surge como resultado del patrón “Reemplazar Datos y Valores por Objetos”. Además, apliqué el patrón “Encapsular atributo”, haciendo que los campos sean private y se acceda a ellos a través de métodos get, siguiendo buenas prácticas de programación orientada a objetos.

Con esto, no solo el código queda mucho más claro, sino que también está preparado para escalar: si más adelante queremos validar el DNI, calcular si el cliente es mayor de edad o cualquier otra lógica, podemos hacerlo dentro de esta clase sin tocar Pedido.

Adjunto código:

*Cliente.java *Pedido.java *Main.java

```java
package entornos.ejercicio1;

/**
 * Clase que representa un Cliente con nombre, edad y DNI.
 */
public class Cliente {
    private String nombre;
    private int edad;
    private String dni;

    /**
     * Constructor para inicializar un cliente.
     * @param nombre Nombre del cliente.
     * @param edad Edad del cliente.
     * @param dni DNI del cliente.
     */
    public Cliente(String nombre, int edad, String dni) {
        this.nombre = nombre;
        this.edad = edad;
        this.dni = dni;
    }

    public String getNombre() {
        return nombre;
    }

    public int getEdad() {
        return edad;
    }

    public String getDni() {
        return dni;
    }
}
```

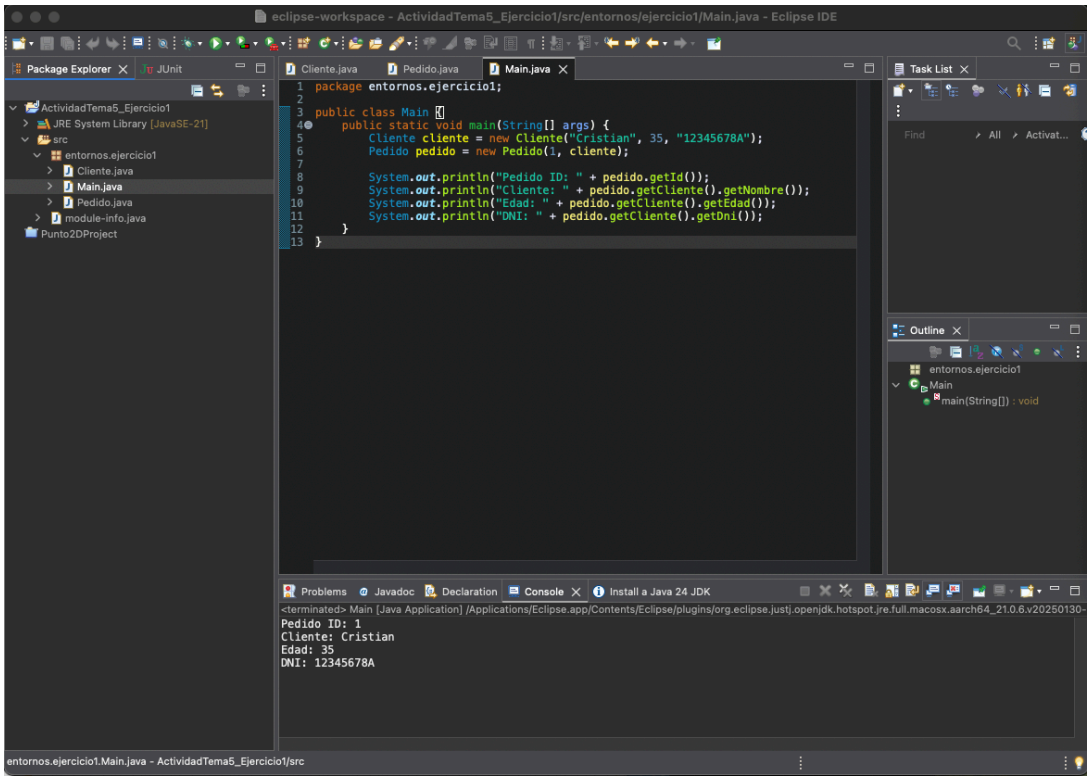
Una vez refactorizado el código, era necesario comprobar que todo funcionaba correctamente. Para eso, creé una clase llamada Main, que incluye el método main() desde donde podemos ejecutar el programa y ver el resultado por consola.

En esta clase simplemente se crea un objeto Cliente con sus datos y luego se usa para crear un objeto Pedido. Finalmente, mostramos los datos por consola para verificar que se han almacenado y accedido correctamente.

Esta parte no implica refactorización en sí, pero es fundamental para probar que los cambios no han roto el comportamiento original del código (algo clave cuando se hace refactoring).

Adjunto código:

Captura con la ejecución del código.



**Adjunto las class junto con la documentación de este ejercicio en el mismo zip.*

EJERCICIO 2

En esta actividad, realizaremos la **documentación de código Java**, usando las etiquetas (tags) del JavaDoc y la herramienta de generación de documentación automática JavaDoc de Eclipse.

Dado el código adjunto abajo, los pasos a realizar son los siguientes:

1. **Crear la clase Stack** (tienes el código abajo)
2. **Comprobar que no tenga errores**. En el caso que tenga, toca solucionarlos**. Comenta que has hecho para solucionarlos** y adjunta una evidencia.
3. **Documentar**:
    a. la clase
    b. cada uno de los campos y métodos
    c. las partes principales del código

    Utiliza las etiquetas y comentarios apropiados. En este enlace podéis encontrar un resumen de los tipos de comentarios Java:

    http://docs.oracle.com/javase/1.5.0/docs/tooldocs/windows/javadoc.html#comments

    Adjunta evidencia del código comentado.

    Es necesario que se vea vuestro nombre como autor del código.

4. **Generad la documentación JavaDoc** con Eclipse, adjuntar evidencia.
5. **Adjuntar** en la entrega el **código Java** correctamente documentado, así como los **documentos JavaDoc generados**

Bloque de código a utilizar en la actividad:

```
public class Stack {

	private int tamaño;
	private Vector<Integer> elementos;

	public Stack() {

		elementos = new Vector<Integer>();
		tamaño = 0;
	}

	public boolean stackVacio () {
		if (tamaño==0) {
			return true;
		}
		return false;
	}

	public void apilar ( Integer o ) {
		elementos.add(tamaño, o);
		tamaño++;
	}

	public Integer desapilar () {
		try {
			if(stackVacio())
				throw new ErrorStackVacio();
			else {
				return elementos.get(--tamaño);
			}
		} catch(ErrorStackVacio error) {
			System.out.println("ERROR: el stack está vacio");
			return null;
		}
	}

	public int getNumElements() {
		return tamaño;
	}

	@SuppressWarnings("serial")
	class ErrorStackVacio extends Exception {
		public ErrorStackVacio() {
			super();
		}
	}

}
```

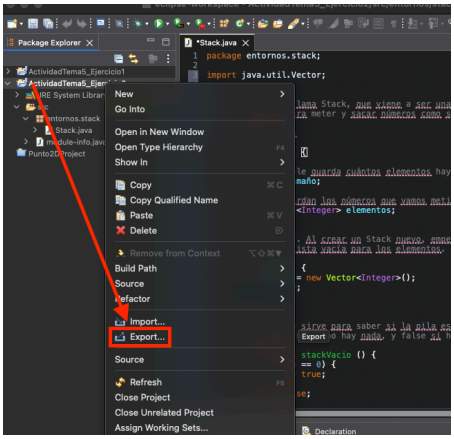
En esta parte de la actividad, el objetivo era aprender a documentar el código en Java utilizando JavaDoc, una herramienta que genera documentación automática en formato HTML a partir de los comentarios que escribimos dentro del código.

Primero, abrí Eclipse y creé un nuevo proyecto llamado ActividadTema5_Ejercicio2, para tener separado este ejercicio del anterior. Dentro de este proyecto, creé un paquete llamado entornos.stack y dentro puse la clase Stack con el código que nos dieron en la actividad.

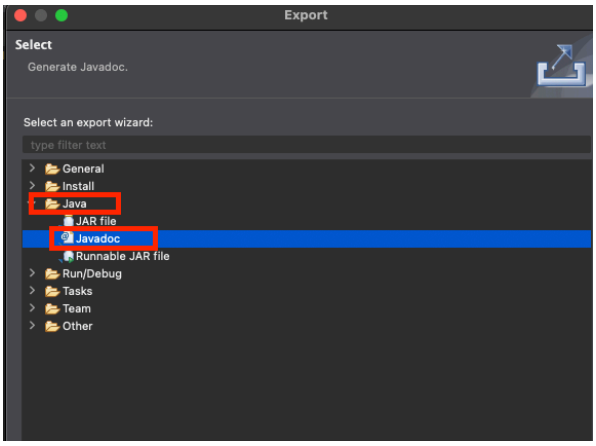
Después de comprobar que el código no daba errores (no tuve que corregir nada raro), empecé a comentar el código usando el formato JavaDoc.

Una vez terminado eso, pasé a generar la documentación JavaDoc. Lo hice desde Eclipse, siguiendo estos pasos:

1. Clic derecho sobre el proyecto → Export > Java > Javadoc.

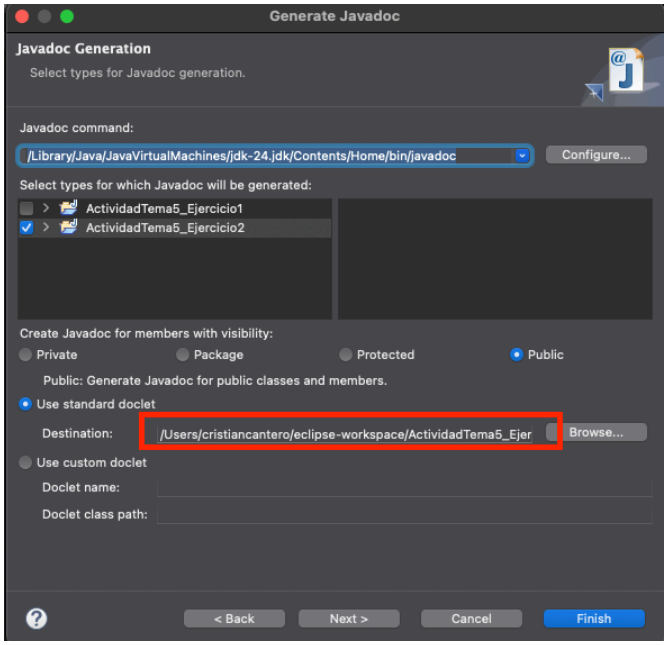


2. Seleccioné la ruta del comando javadoc (que está dentro del JDK).



Linkia FP

3. Elegí la carpeta donde quería que se guardara la documentación (por defecto se crea en una carpeta llamada doc dentro del proyecto).



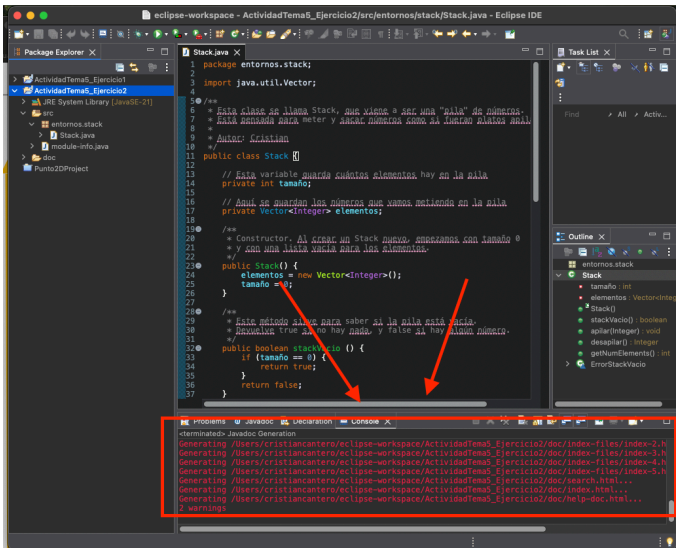
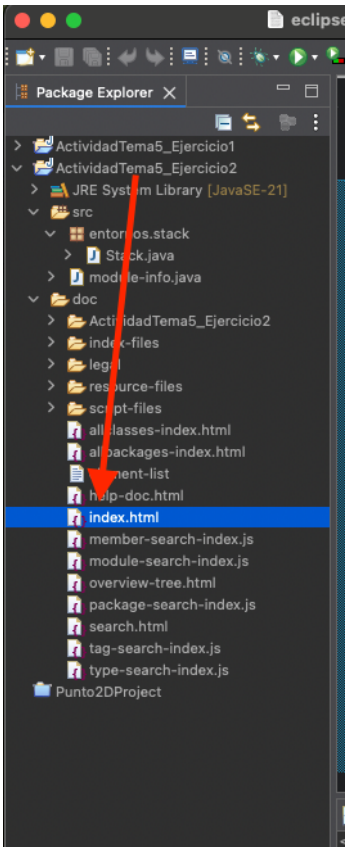
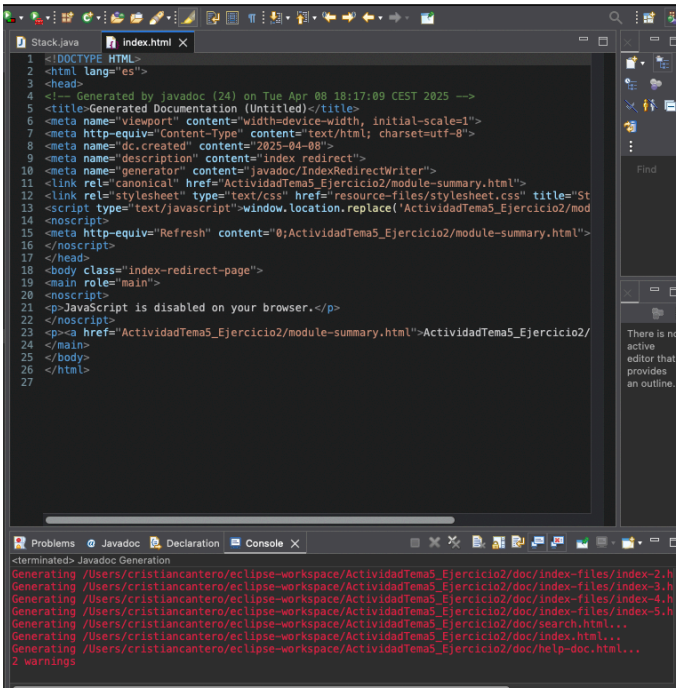
4. Pulsé Finish.

En la consola apareció el mensaje de que se estaba generando la documentación. Al final me salieron dos warnings, pero no son errores ni nada grave, solo son avisos de que algo podría estar más completo. Aun así, la documentación se generó correctamente.

Después abrí el archivo index.html y pude ver toda la documentación generada en formato web. Ahí se puede navegar por la clase, ver qué hace cada método y todo eso.

A continuación dejo las capturas de:

- La consola de Eclipse mostrando la generación del JavaDoc.



- El archivo index.html abierto en el navegador con la documentación generada.